RESEARCH AND INTEGRATION REVIEW

Further Lean proof simplifications

Four additional directions after the earlier proposals

Prepared for Vladimir Reshetnikov

26 October 2026

Scope. The maintained Lean sources in `Computability/DiophantineConsistency`, reviewed at commit `e1c8914bc7064d9a8e7c06171a18069a949badd0`. Lean 4.32.0 and the repository's pinned mathlib revision are used for the supplied checks.

Main result. Replace three independently checked modular tables for the literal kernel by one certificate modulo a common multiple. The proposed source file shrinks from 843 to 426 physical lines, including the new helper and the retained public interface. Three smaller additions simplify iterated exponent reduction, point updates of packed words, and the reflected contribution at the origin.

Exclusion baseline. All ideas in the 23 archived rounds a–w and the five current incoming ZIPs are treated as already applied. The report explains the remaining contribution of each proposal and records rejected overlaps.

Validation boundary. The supplied arithmetic cores, exact-data certificates and scoped proof checks have been compiled. Proposed production patches have been prepared against the pinned source. The full production import graph and downstream clients have not been rebuilt in this review. These are reviewable integration candidates, with explicit evidence rather than an assertion of a completed repository migration.

Source, patches, replay instructions, audit receipts and validation logs accompany this report. No change to the mathematical consistency statement, literal kernel or executable encoding is proposed.

Contents

1 Results and the baseline they improve **2**
1.1 What “assume already applied” means here . . . 2
1.2 Source identification . . . 3

2 A: certify the kernel once in a common residue ring **3**
2.1 The present repetition . . . 3
2.2 The certificate and its projection . . . 3
2.3 Preserving the literal kernel . . . 4
2.4 Size and cost . . . 4

3 B: replace exponent-specific arguments by a power fixed point **5**
3.1 The reusable mathematical statement . . . 5
3.2 Application to the actual controller . . . 5
3.3 What becomes simpler . . . 5

4 C: treat a point mutation as a finite weighted update **6**
4.1 An equality over natural numbers . . . 6
4.2 The packed-word specialization . . . 6
4.3 The boundary that must remain . . . 7

5 D: reuse the reflected-source theorem at offset zero **7**
5.1 The existing theorem already contains the origin case . . . 7
5.2 Preserved generality . . . 7

6 A useful synthesis: one controller-unit certificate **8**

7 Non-repetition ledger **9**

8 Validation and practical integration **9**
8.1 What was checked . . . 9
8.2 Recommended order . . . 10
8.3 Interpreting the metrics . . . 10

9 Further research questions **11**

10 Artifact guide and source references **11**

1 Results and the baseline they improve

The strongest opportunity is a reduction in the number of certificates that must be maintained. `CurrentKernelResidues2011` currently checks the same 941-entry selector polynomial separately in three residue rings. The proposed replacement checks it once modulo

$$M = 47 \cdot 2011 \cdot 120199 = 11\,360\,848\,883$$

and maps that result to the three existing targets. This change retains the original physical-order bridge and the same literal kernel. It adds no assertion that a finite test proves a general semantic property.

The other proposals address independent obligations that remain after the earlier work: an elementary fixed-point law for a power map; the exact effect of a point update on a finite weighted sum; and a boundary instance of an already established reflected-source theorem.

Proposal	Maintained target and additional contribution	Net lines
A	`CurrentKernelResidues2011`: one common-modulus certificate replaces three checked tables	417
B	`CurrentControllerUnits2011`: one elementary power-iteration proof replaces order/Frobenius branches	5
C	`CurrentDummyMutation2011`: finite-sum update and pointwise bound APIs	8
D	`CurrentWordOrigin2011`: specialize the existing reflected-source result at offset zero	7

Table 1: Measured physical source reductions against the pinned maintained files, including the helpers introduced by each production patch. These are not measurements of a hypothetical source tree after every earlier proposal has been implemented.

The patches together remove 437 physical lines from four current files. This total is a source comparison, not a runtime estimate. In particular, arithmetic modulo a larger integer can cost more even when there are fewer certificates. No build-time speedup is claimed.

1.1 What “assume already applied” means here

The prior corpus is much broader than the adoption record alone. It includes proposals that were not adopted, design-only sections, abandoned implementations, supporting notes and pending ZIP submissions. A proposal is excluded here if its mechanism has already been proposed, even when the maintained tree still contains the longer proof.

The following are therefore background infrastructure, not contributions of this report: generated circuit steps, scanner folds and closure invariants, representation naturality, witness layouts, shared residual-sum lemmas, polynomial coefficient compilation, finite table chunking, constructor views, closed carry intervals, ordered-semiring dependency rearrangement, period lattices, helical coordinates and edge-field constructions. Stronger-to-weaker congruence transport is also already available in this baseline. Proposal A uses that fact; its new step is to consolidate the independent certificates to which it is applied.

There is no claim that the mathematical facts used here are new. The contribution is a source-specific, audited set of additional proof simplifications. The exclusion audit establishes scoped non-repetition, not a global novelty theorem.

1.2 Source identification

The snapshot was obtained through the GitHub connector and corroborated locally. All 690 files in the maintained Lean subtree were compared with the Git blob identifiers returned for the pinned tree. All matched. The audit also checked every one of the 1,286 files under `research/lean-proof-simplification`, and all five incoming ZIPs. SHA-256 hashes were recorded for all 1,042 decompressed archive members.

Complete byte coverage does not mean that every copied baseline or old compiler log was read line by line. Principal proposal sections from all 23 rounds and all five incoming reports were read; supporting notes, cumulative exclusion ledgers and prototype files were inspected and searched for the mechanisms under consideration. The receipts distinguish these two kinds of coverage.

2 A: certify the kernel once in a common residue ring

2.1 The present repetition

The maintained module defines a high-to-low evaluator on the literal ordered selector rows:

$$E_r([]) = 0, \qquad E_r((i, w) :: l) = 2w\, r^{|l|} + E_r(l).$$

The first coordinate of a row is retained as part of the original table; the evaluation uses its weight. The exact source has 59 chunks containing 941 rows in total. Each chunk is checked separately at moduli 47, 2011 and 120199. That creates 177 chunk equalities, followed by three assembly arguments and three control calculations.

Earlier proposals already supply generic chunk assembly and homomorphic evaluation. Applying those ideas does not remove the need to maintain three separate numerical certificates. The additional step is to choose one modulus that projects onto every target and to generate a single set of chunk values there.

2.2 The certificate and its projection

Let $\rho = 2^{47}$, and let M be the common multiple above. The exact computations in the supplied checked fixture are:

Expression	Least nonnegative residue modulo M
ρ	10,653,241,607
E_ρ(941 literal rows)	10,463,387,658
34 packed control terms	4,297,442,889
Whole reflected-kernel expression	1,491,904,576

Table 2: Values established by the exact-data Lean fixture. The production bridge from the literal `K` to this polynomial expression is preserved unchanged in the patch.

For each $p \in \{47, 2011, 120199\}$, divisibility $p \mid M$ gives a ring homomorphism

$$\pi_p : \mathbb{Z}/M\mathbb{Z} \longrightarrow \mathbb{Z}/p\mathbb{Z}.$$

Applying it to the common equality gives exactly the existing endpoint values:

$$K \equiv 26 \pmod{47}, \qquad K \equiv 1995 \pmod{2011}, \qquad K \equiv 114787 \pmod{120199}.$$

This argument needs no Chinese remainder reconstruction. The projections require divisibility alone; pairwise coprimality is not a premise of the transport lemma. A common multiple, rather than necessarily the product, is the general interface.

At the evaluator level, the reusable law is

$$\phi(E_r(l)) = E_{\phi(r)}(l)$$

for a semiring homomorphism ϕ. It follows by induction on the list and preservation of natural-number casts, multiplication, addition and powers. This homomorphism law is supporting infrastructure. The principal reduction is 177 numerical block proofs becoming 59.

2.3 Preserving the literal kernel

The maintained `rawWord_cast` theorem identifies the actual kernel expression as

$$\begin{aligned} K = {} & \rho^{18} + \rho^{1004} + \rho^{2020} + 2\, w_{\mathrm{Start}} \rho^{1005} \\ & + E_\rho(\mathrm{rows})\rho^{64} + C_\rho, \end{aligned}$$

where $w_{\mathrm{Start}} = 34\,876\,427\,265$, and C_ρ is the sum of the 34 existing packed color controls. The patch preserves the definitions, the selector permutation theorem, `selector_reflection`, and `rawWord_cast`. It changes the numerical certificate layer and adds a common-modulus endpoint.

The standalone fixture contains byte-identical definitions of all 59 chunks from `WangTiles942PhysicalData`. A Python audit confirms both the literal definitions and the count of 941 row pairs. The fixture checks the reflected expression, including the protected Start term, all three bare terms and all 34 control terms. This is considerably stronger evidence than checking a separately typed decimal value for `K`.

It is still important to state the remaining integration boundary accurately: the full production dependency graph that proves the physical permutation and binds `rawWord_cast` was not rebuilt. Its source is unchanged, and the patch's endpoints are reviewable, but the isolated fixture is not a substitute for that final module build.

2.4 Size and cost

The current file has 843 physical lines. The candidate has 426, a reduction of 417 lines after including the common certificate, the projection helper and the retained public wrappers. All original public declaration headers are preserved; the common-modulus theorem is an additional declaration. The exact patch and metrics are delivered.

The common modulus is larger than each individual modulus. Proof term size and arithmetic cost can consequently increase per block. A sensible implementation keeps the existing bounded chunks, profiles them under the pinned compiler, and adjusts chunk size only after observing a concrete problem. Replacing kernel-checked arithmetic with `native_decide`, hiding a new trust assumption, or claiming a speedup from line counts would not be justified.

3 B: replace exponent-specific arguments by a power fixed point

3.1 The reusable mathematical statement

For any monoid, any element x, and any natural numbers a, n,

$$x^a = x \quad \Longrightarrow \quad x^{a^n} = x.$$

The proof is an induction on n. The zero case is $x^1 = x$. For the successor,

$$x^{a^{n+1}} = x^{a^n a} = (x^{a^n})^a = x^a = x.$$

It does not require commutativity, a field, a finite carrier, a multiplicative-order calculation or primality.

The checked Lean core is:

```
theorem pow_pow_fixed {M : Type*} [Monoid M]
    (x : M) (a n : ℕ) (h : x ^ a = x) :
    x ^ (a ^ n) = x := by
  induction n with
  | zero => simp
  | succ n ih => rw [pow_succ, pow_mul, ih, h]
```

3.2 Application to the actual controller

The existing definitions are

$$a = 120199, \qquad B = (2^{47})^{2011} = 2^{94517}, \qquad \text{dimension}(n) = a^n, \quad q(n) = B^{a^n}.$$

The current `power_dimension_47` and `power_dimension_2011` proofs reason separately through the orders 23 and 402. The auxiliary-modulus proof invokes Frobenius. All three have the same target: reducing B^{a^n} back to B in a fixed residue ring.

The conservative production patch proves the three small seeds

$$37^{120199} = 37 \text{ in } \mathbb{Z}/47\mathbb{Z}, \quad 1283^{120199} = 1283 \text{ in } \mathbb{Z}/2011\mathbb{Z}, \quad 37435^{120199} = 37435 \text{ in } \mathbb{Z}/120199\mathbb{Z}$$

and uses the uniform monoid lemma. These are the already established residues of B, so B can remain opaque throughout the numerical check. The delivered base proposal uses three small seed checks; it does not pretend that one combined seed has replaced them there. Section 6 discusses the combined alternative separately.

The original theorem statements remain unchanged. Both independent exponents may still be zero. In particular, $a^0 = 1$, so the $n = 0$ case must not be silently removed by a positive-exponent hypothesis. The generic core also handles $a = 0$ correctly: its hypothesis then forces $x = 1$.

3.3 What becomes simpler

The net measured reduction is only 5 physical lines, including the new shared helper and the import change. The larger benefit is a uniform explanation of all three theorems. Future modulus changes require a fixed-point seed, rather than a choice between a multiplicative-order argument and a finite-field argument.

The replacement proof no longer uses `Mathlib.FieldTheory.Finite.Basic` directly, so that direct import is removed in the candidate. This is not a claim that the entire transitive dependency graph loses finite-field code. Nor does the patch delete the existing order theorems: other clients can continue to use them.

This is distinct from earlier suggestions about stronger congruences or general semiring evaluation. Its new invariant is stability under repeated application of a power map. It also supplies a simple explanation for why the controller dimensions preserve the relevant residues.

4 C: treat a point mutation as a finite weighted update

4.1 An equality over natural numbers

For a finite index type I, arbitrary functions $c, w : I \to \mathbb{N}$, an index i, and a natural increment δ,

$$\sum_{j \in I} \mathrm{update}(c, i, c(i) + \delta)(j) w(j) = \sum_{j \in I} c(j) w(j) + \delta w(i).$$

This statement does not require bounded digits, positivity of the radix, a carry invariant or a modular nonwrapping assumption. It is an ordinary natural-number identity.

The proof splits the finite sum into its selected summand and the common complement. The complement is identical before and after the update. Distributing multiplication in the selected summand completes the argument. Using the library update API expresses exactly this reasoning:

```
simp_rw [Function.apply_update (fun j v => v * w j)]
rw [Finset.sum_update_of_mem (Finset.mem_univ i),
  Finset.sum_eq_add_sum_sdiff_singleton_of_mem
    (Finset.mem_univ i) (fun j => c j * w j)]
ring
```

This avoids first proving an equality of whole functions with a separate case split at every index, rewriting the packed word as a sum of two words, and then evaluating an indicator word. Earlier linearity and point-mass convolution APIs remain assumed available; the additional simplification is the direct finite-update application at this actual mutation theorem.

4.2 The packed-word specialization

For $I = \mathbb{Z}/n\mathbb{Z}$, let

$$\mathrm{word}_\rho(c) = \sum_i c(i) \rho^{\mathrm{val}(i)}.$$

Choose $w(i) = \rho^{\mathrm{val}(i)}$ and $\delta = 1$. The exact target is the existing `word_update_increment` theorem in `CurrentDummyMutation2011`. The production replacement expands only `word`, uses the update/sum APIs, and closes the remaining polynomial identity.

The standalone file also proves the general δ theorem. The production patch does not need to add that helper merely to obtain a short special case; its source metric includes only the actual changes proposed for the maintained file. This keeps helper overhead visible.

A companion local change replaces the manual cases in `update_boolean` by `Function.forall_update_iff`: the new value satisfies the bound, and all other values inherit it. It is part of the same update API simplification.

4.3 The boundary that must remain

The ordinary packed-word identity is separate from the later statement about the normalized transport word. The latter still needs the explicit nonwrapping bound

$$F + A\delta < \text{modulus}$$

and the strict bound on the new representative. The patch does not remove these hypotheses from `transport_increment`, `normalized_increment` or `rank_decrement`.

This matters because a modular increment can wrap even when the underlying source update is a perfectly valid natural-number sum. A shorter proof of the source identity cannot justify a stronger normalized-word theorem.

5 D: reuse the reflected-source theorem at offset zero

5.1 The existing theorem already contains the origin case

`CurrentReflectedGeometry2011.reflected_source` describes a reflected source within an aligned cell. With $h \le 1004$, $p \le 985$, and the established periodic support condition, it gives

$$c\big(b + h - (1005 - p)\big) = \begin{cases} c\big(b + (h + p - 1005)\big), & 1005 \le h + p, \\ 0, & \text{otherwise.} \end{cases}$$

All address equalities are in the actual cyclic word. The theorem already deals with wrapped negative offsets by the support argument.

At the origin, $h = 0$. Since $p \le 985 < 1005$, the condition in the first branch is false. The desired source is zero immediately. No reconstruction of the preceding cell is needed.

The existing `origin_reflected_zero` proof instead writes the address as a high field in the previous cell, proves the corresponding base-shift identity, normalizes the natural casts, and invokes `word_zero_high`. The replacement specializes `reflected_source` at zero and simplifies its false branch. It introduces no helper and preserves the theorem's full interface.

5.2 Preserved generality

The source word may have arbitrary natural-valued dummy and marker controls. Boolean hypotheses are not introduced. The proof still includes the one-cell case $N = 1$ and the wrap from the first cell to the last. It relies on the same full cyclic modulus $N \cdot 2011$, rather than replacing that modulus by a coprime quotient.

At the level of the full reflected sum, the existing `reflected_zero_before_twenty` theorem has the same useful specialization. The production review keeps the public origin theorem available; callers can reuse either the pointwise wrapper or the stronger sum theorem. The reported patch savings concern the exact retained public proof, rather than counting deletion of an API that clients may still use.

This is a small endpoint reuse, not a proposal for another general geometry framework. Earlier proposals about correcting the origin digit of a packed mask concern a different assertion. Earlier cyclic-coordinate and support abstractions remain part of the assumed baseline.

6 A useful synthesis: one controller-unit certificate

The common-modulus and power-fixed-point proposals fit together. This section records their additional combined consequence, rather than counting it as an independent fifth idea.

The common residue of the actual base is

$$B \equiv 10\,287\,028\,452 \pmod{M},$$

and this residue is fixed by the 120199-th power map. Thus both $q(n)$ and $B^{120199^{r_w}}$ reduce to the same residue, for arbitrary independent natural exponents n, r_w.

The actual controller quantity is

$$\Gamma(r_w, n) = \rho^{977}\big(q(n)^2 - 1\big)\big(1 + q(n)(K + B^{120199^{r_w}})\big).$$

Substituting the common B and K residues yields

$$\Gamma(r_w, n) \equiv 706\,142\,772 \pmod{M}, \qquad \gcd(706\,142\,772, M) = 1.$$

A single coprimality certificate can therefore feed the existing coprimality conclusions for 47, 2011 and 120199, and then their required powers.

Natural subtraction deserves explicit attention: the cast of $q(n)^2 - 1$ to a residue ring is valid in the displayed form only after proving $1 \le q(n)^2$. The combined fixture retains this premise rather than treating natural subtraction as ring subtraction unconditionally. The actual source already proves the needed inequality from $q(n) > 1$.

The delivered combined prototype makes the B, K, and radix residues explicit inputs and checks the modular algebra and gcd result. Its residue assumptions are connected to the separate certificates and the unchanged source bridge. It is not presented as a fully integrated replacement for every production `Gamma` theorem. No additional line-saving total is assigned to this optional synthesis.

7 Non-repetition ledger

Candidate	Closest earlier ideas, treated as applied	Additional contribution retained here
A	a/g: stronger-to-weaker congruences; b: chunk assembly; f: homomorphic coefficient evaluation	Check the 941-row kernel table once in a common residue ring instead of retaining three independent checked tables.
B	General algebraic interfaces and numerical certificate infrastructure	The invariant $x^a = x$ is propagated under exponent iteration, replacing different order and Frobenius proof routes for the same actual controller quantity.
C	Word additivity and point-mass convolution	Apply the finite update/selected-summand APIs directly to the existing mutation theorem, with ordinary natural arithmetic and unchanged transport hypotheses.
D	Cyclic coordinates, support interfaces, and packed-mask origin corrections	Specialize an already stronger reflected-source theorem at $h = 0$, eliminating the separate previous-cell address derivation.

Several tempting directions were explicitly rejected. Deriving a stored carry recurrence from decomposition and digit bounds, then restoring the old dot notation as a theorem, is already proposed in round b. Moving representation-size constructor laws earlier is also in round b. Termwise-minimum/equality saturation for residual sums is in round q. A proposed one-line list-preimage rewrite was discarded after the suggested library theorem name was not found in the pinned API. None of these is included as a new result or counted in the savings.

The rejection ledger is useful because code search alone is insufficient. Some prior ideas occur only in supporting findings files or in a design paragraph, not in the summary tables or an adopted patch.

8 Validation and practical integration

8.1 What was checked

The supplied validation receipt identifies the exact files, hashes, compiler version, commands, exit statuses and axiom reports. The checks and their precise scopes are listed below. Candidate production modules remain separate from these successful isolated checks:

1. The common-modulus fixture with the exact 59 literal chunk definitions, all 941 selector rows, 34 controls, reflected-kernel expression and three projections.
2. The generic power-fixed-point law, the three exact numerical seeds and the source-shaped power-reduction statements. Additional concrete before/after fixtures are supplied for replay, but their local runs stalled in file I/O before producing diagnostics. They are explicitly not counted as successful checks; the checked power artifact uses symbolic B with the existing residue hypotheses.

3. The general weighted-update identity, its exact packed-word specialization and the pointwise update-bound replacement.
4. The origin specialization, with the existing support and reflected-source lemmas extracted into an isolated namespace, and a generic reflected-sum consequence.
5. The combined controller-residue calculation and coprimality certificate, with every bridge assumption explicit.

The new delivered proofs contain no `sorry`, `admit`, custom axiom declaration or `native_decide`. Axiom output is retained so that ordinary Lean foundations such as propositional extensionality, choice and quotient soundness remain visible. An isolated check can show that a new proof term is accepted without establishing that every unchanged production dependency was rebuilt.

The source patches are applied to temporary copies for a clean applicability check. This verifies exact patch targeting and preservation of the intended source edits; it is not a Lean build. The complete production modules and their clients remain the final integration check. The source-equivalence and byte-integrity scripts are likewise distinguished from theorem proving.

8.2 Recommended order

The smaller endpoint changes can be reviewed independently. For the controller work, apply the common-modulus certificate before using its new endpoint elsewhere. Apply the power-fixed-point patch separately so that its benefit and any dependency change remain clear. Treat the combined Γ prototype as a follow-on port with its own complete module check.

From a checkout of the pinned repository, the relevant production checks are:

```
lake build DiophantineConsistency.CurrentKernelResidues2011
lake build DiophantineConsistency.CurrentControllerUnits2011
lake build DiophantineConsistency.CurrentDummyMutation2011
lake build DiophantineConsistency.CurrentWordOrigin2011
```

The accompanying replay script runs the isolated modules in a compatible mathlib environment. It does not silently claim to run these production commands. Preserve the repository's pinned toolchain and dependency revisions when comparing results.

Before accepting the combined controller port, check the existing public residues and coprimality statements, independent zero exponents, arbitrary control maps where allowed, the $N = 1$ word case and all nonwrapping premises. These are concrete boundaries of the changed proofs, not hypothetical extra requirements.

8.3 Interpreting the metrics

Physical lines include comments and blank lines; machine-readable records also retain additional code-oriented counts where available. The measurement baseline is the current pinned maintained source. A virtual source tree with every earlier proposal applied does not have a uniquely determined layout, because some earlier packets offer competing implementations. Its net line count cannot be honestly asserted.

The retained proposals address obligations that survive those earlier abstractions. The 177-to-59 certificate count is especially robust under that interpretation. The exact current-source reductions are provided to make the patches concrete, not to double-count savings from older proposals.

9 Further research questions

What is the best common certificate granularity? A single common modulus reduces duplication, but the optimum chunk length may differ from the previous 16-row bound. Compare total verification time and object size on identical source and hardware before changing the chunk generator. A vector of residues may have different performance from a scalar common modulus, while representing the same maintenance decision.

Can the combined Γ certificate replace the three remaining proof chains cleanly? The modular algebra is supplied. A useful next port would preserve the public wrappers, derive their coprimality from the common certificate, and measure the final total including the shared infrastructure. The natural-subtraction side condition and all independent exponent cases must remain explicit.

Can more mutable controls use the same update theorem? The point-update law handles arbitrary natural increments and weights. Multiple updates at distinct coordinates can be composed using existing update commutation, but a batch API is worthwhile only when actual clients remove more code than the helper introduces. A speculative new update framework is not needed for the present patch.

Where do other boundary proofs duplicate a stronger theorem? The origin case suggests an audit tactic for future human review: look for a theorem proved by reconstructing a special address, and test whether an existing interval theorem contains that endpoint. Such instances must be checked individually; generic support and coordinate redesigns have already been proposed.

Can the unchanged physical-order bridge become cheaper to replay? The large certificate does not weaken that obligation. Separating its expensive rebuild from numeric certificate checks could improve development feedback, provided the final production build still binds every numerical statement to the literal data and original kernel.

10 Artifact guide and source references

The ZIP includes the report PDF and this complete LaTeX source, the four proposed patches, standalone Lean checks, targeted source snapshots, source and exclusion receipts, compiler logs, exact source metrics, replay scripts and a checksum manifest. `README.txt` gives commands and clearly identifies candidate patches versus checked isolated modules.

The snapshot and original declarations can be inspected through these pinned source links:

[1] Maintained DiophantineConsistency Lean subtree.
[2] CurrentKernelResidues2011: evaluator, chunk certificates and K endpoints.
[3] WangTiles942PhysicalData: the literal ordered chunks.
[4] CurrentControllerUnits2011: dimensions, power reductions and unit conclusions.
[5] CurrentDummyMutation2011: source update and transport increment.
[6] CurrentWordOrigin2011: origin support and normalized digit proofs.
[7] CurrentReflectedGeometry2011: the stronger reflected-source theorem.
[8] Excluded rounds a–w, and five excluded current incoming ZIPs.
[9] Pinned Lean toolchain and dependency manifest: Lean 4.32.0, mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`.